Heap Sort

Heap Sort is an efficient sorting technique based on the **heap data structure**.
The heap is a nearly-complete binary tree where the parent node could either be minimum or maximum. The heap with minimum root node is called **min-heap** and the root node with maximum root node is called **max-heap**. The elements in the input data of the heap sort algorithm are processed using these two methods.

The heap sort algorithm follows two main operations in this procedure −

- Builds a heap H from the input data using the **heapify** (explained further into the chapter) method, based on the way of sorting – ascending order or descending order.
- Deletes the root element of the root element and repeats until all the input elements are processed.

Heap Sort Algorithm

The heap sort algorithm heavily depends upon the heapify method of the binary tree. So what is this heapify method?

Heapify Method

The *heapify* method of a binary tree is to convert the tree into a heap data structure. This method uses recursion approach to heapify all the nodes of the binary tree.

Note − The binary tree must always be a complete binary tree as it must have two children nodes always. The complete binary tree will be converted into either a max-heap or a min-heap by applying the *heapify* method.

Heap Sort Algorithm

As described in the algorithm below, the sorting algorithm first constructs the heap ADT by calling the Build-Max-Heap algorithm and removes the root element to swap it with the minimum valued node at the leaf. Then the heapify method is applied to rearrange the elements accordingly.

```
Algorithm: Heapsort(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
exchange A[1] with A[i]
A.heap-size = A.heap-size - 1
MAX-HEAPIFY(A, 1)
```

Analysis

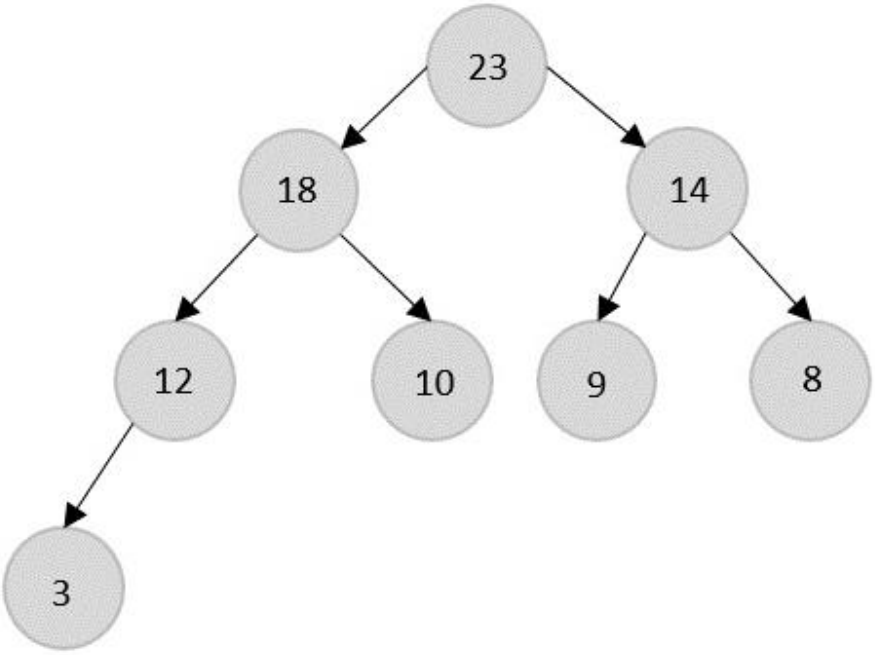
The heap sort algorithm is the combination of two other sorting algorithms: insertion sort and merge sort. The similarities with insertion sort include that only a constant number of array elements are stored outside the input array at any time.
The time complexity of the heap sort algorithm is **O(nlogn)**, similar to merge sort.

Example

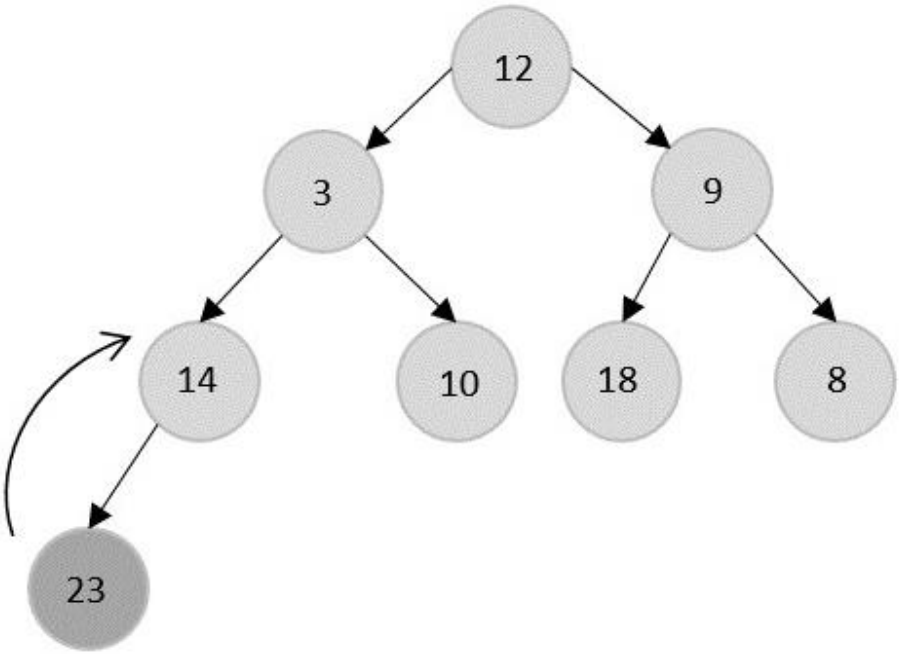
Let us look at an example array to understand the sort algorithm better −

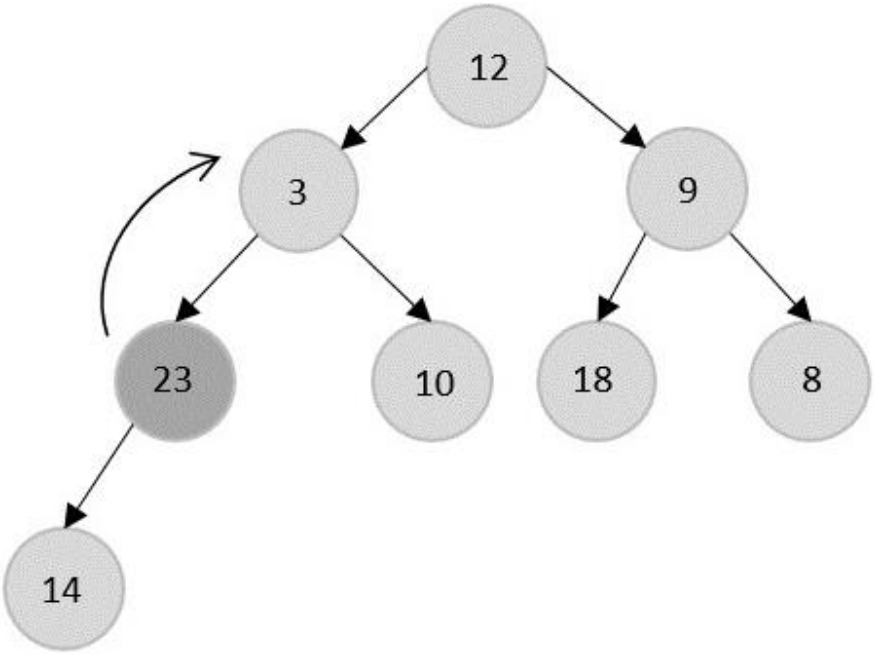
12 3 9 14 10 18 8 23

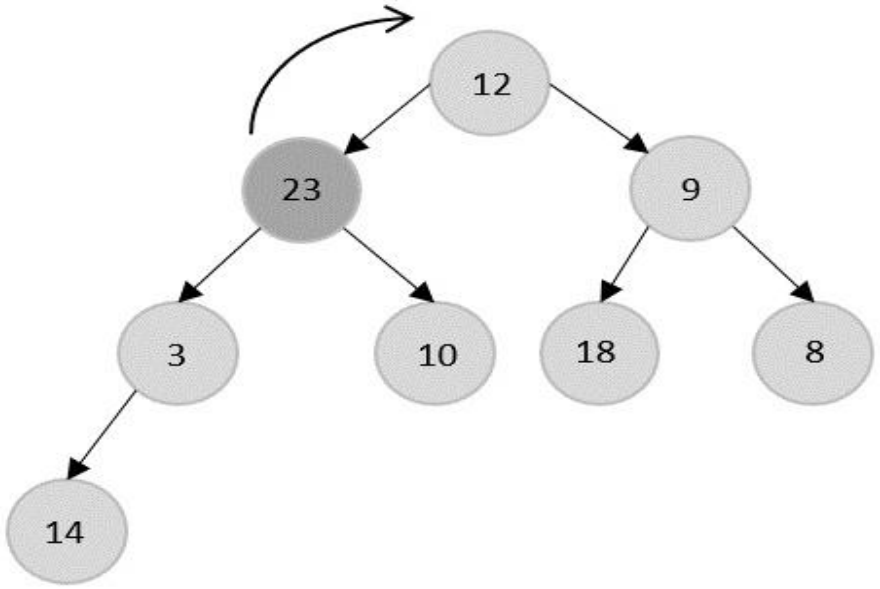
Building a heap using the BUILD-MAX-HEAP algorithm from the input array −

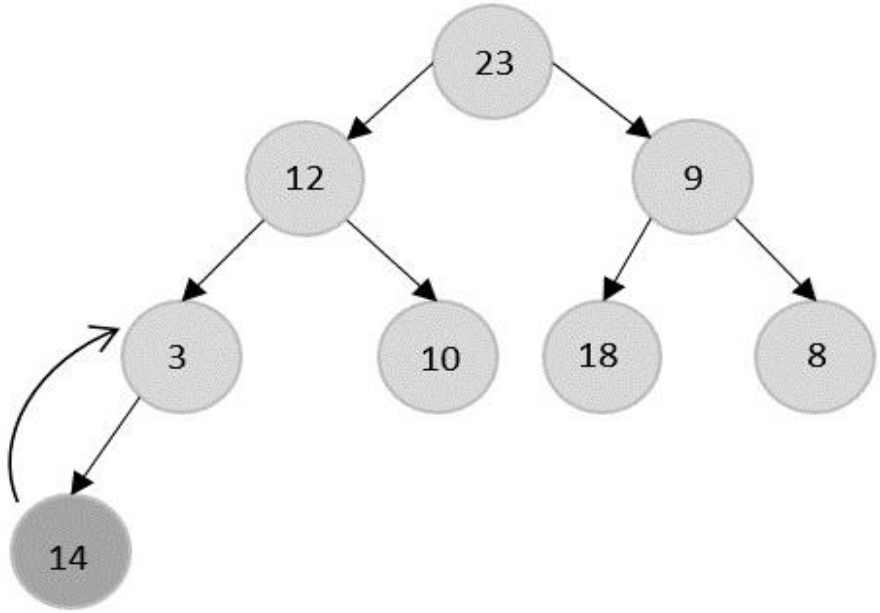


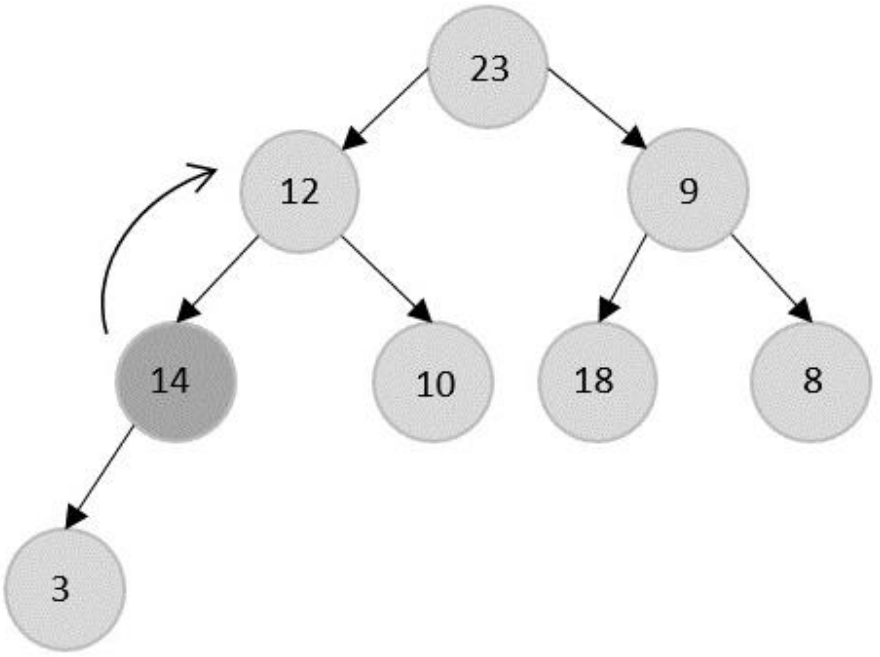
Rearrange the obtained binary tree by exchanging the nodes such that a heap data structure is formed.

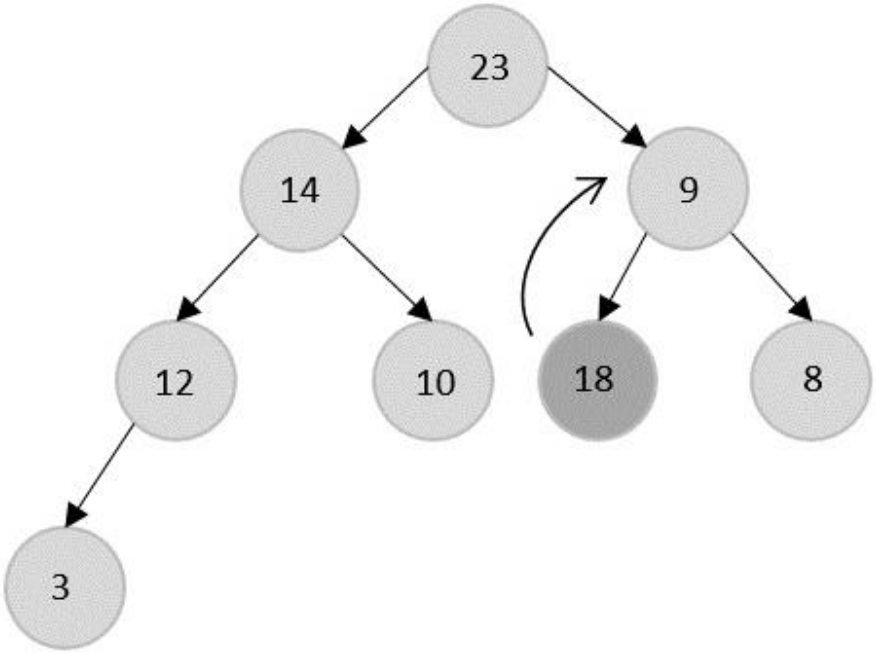


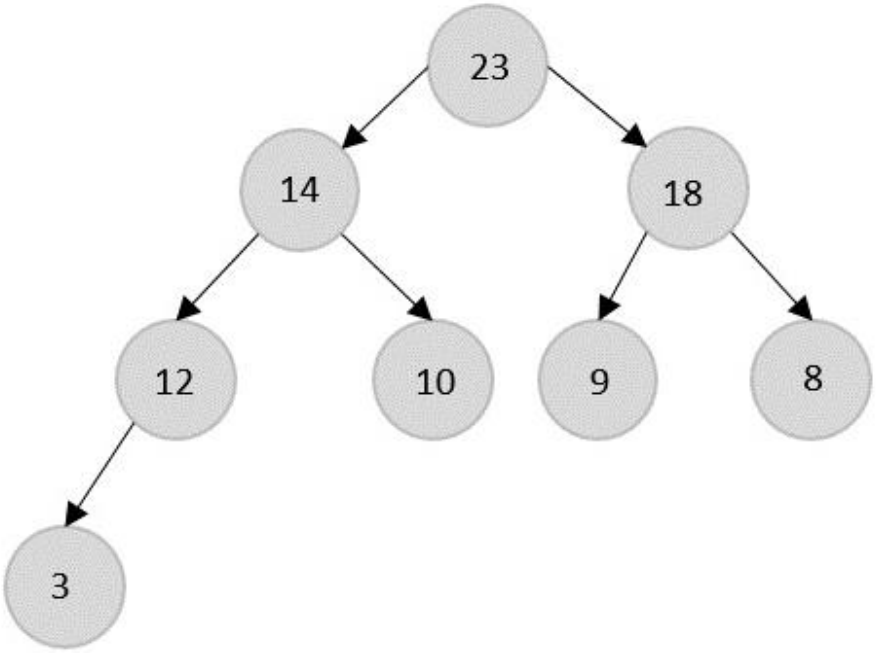



12
23
9
3
10
18
8
14


23
12
9
3
10
18
8
14


23
12
9
14
10
18
8
3

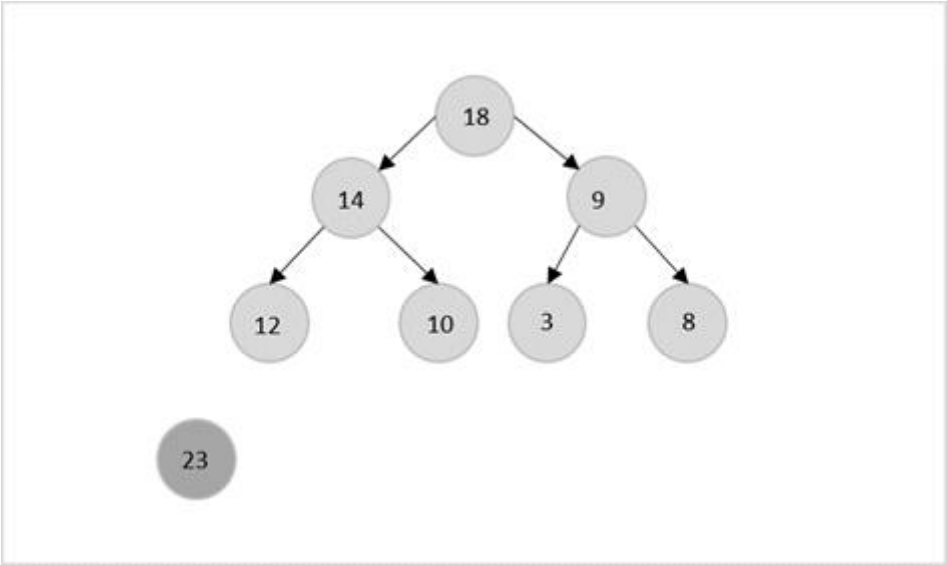




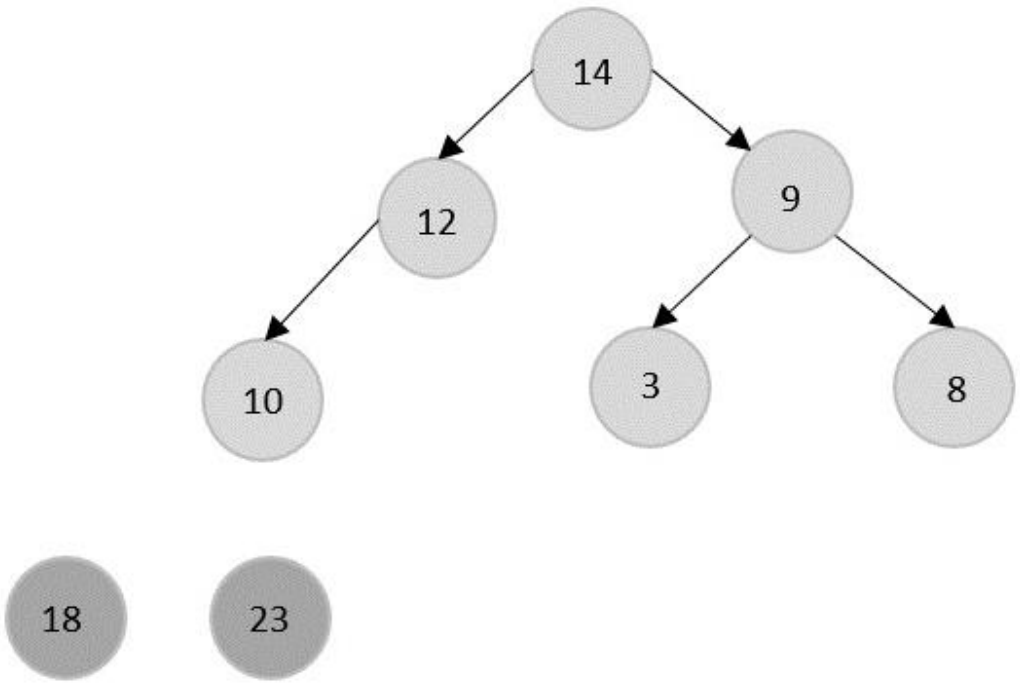
The Heapsort Algorithm

Applying the heapify method, remove the root node from the heap and replace it with the next immediate maximum valued child of the root.

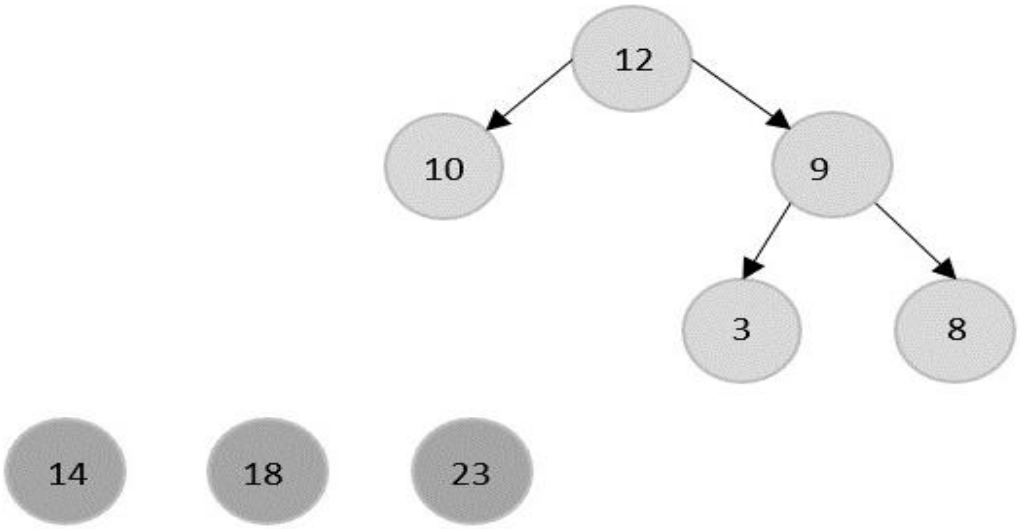
The root node is 23, so 23 is popped and 18 is made the next root because it is the next maximum node in the heap.



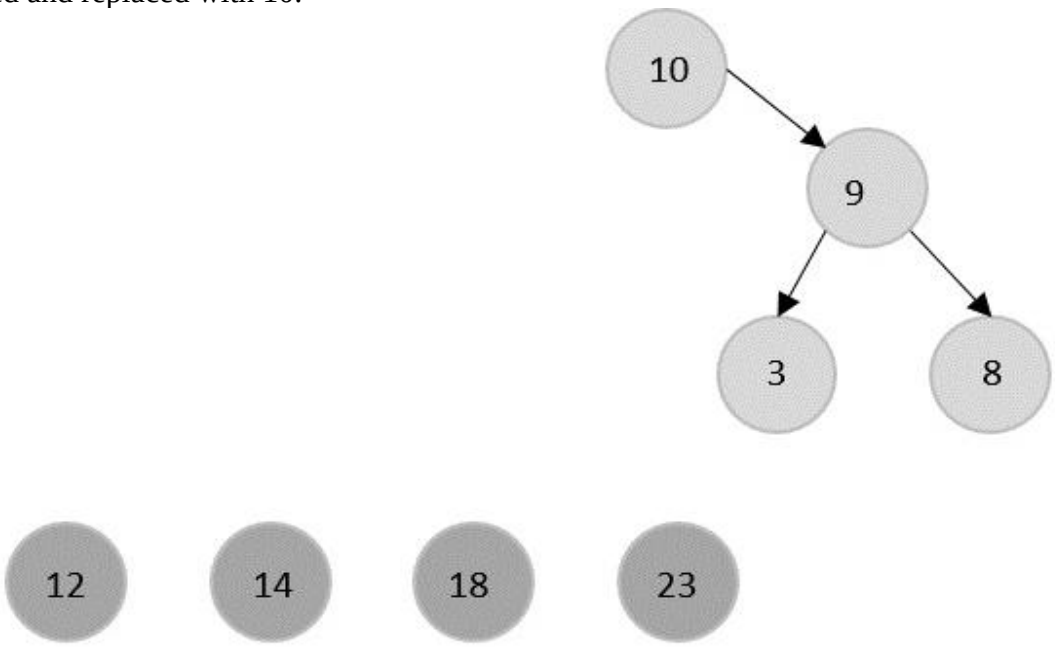
Now, 18 is popped after 23 which is replaced by 14.



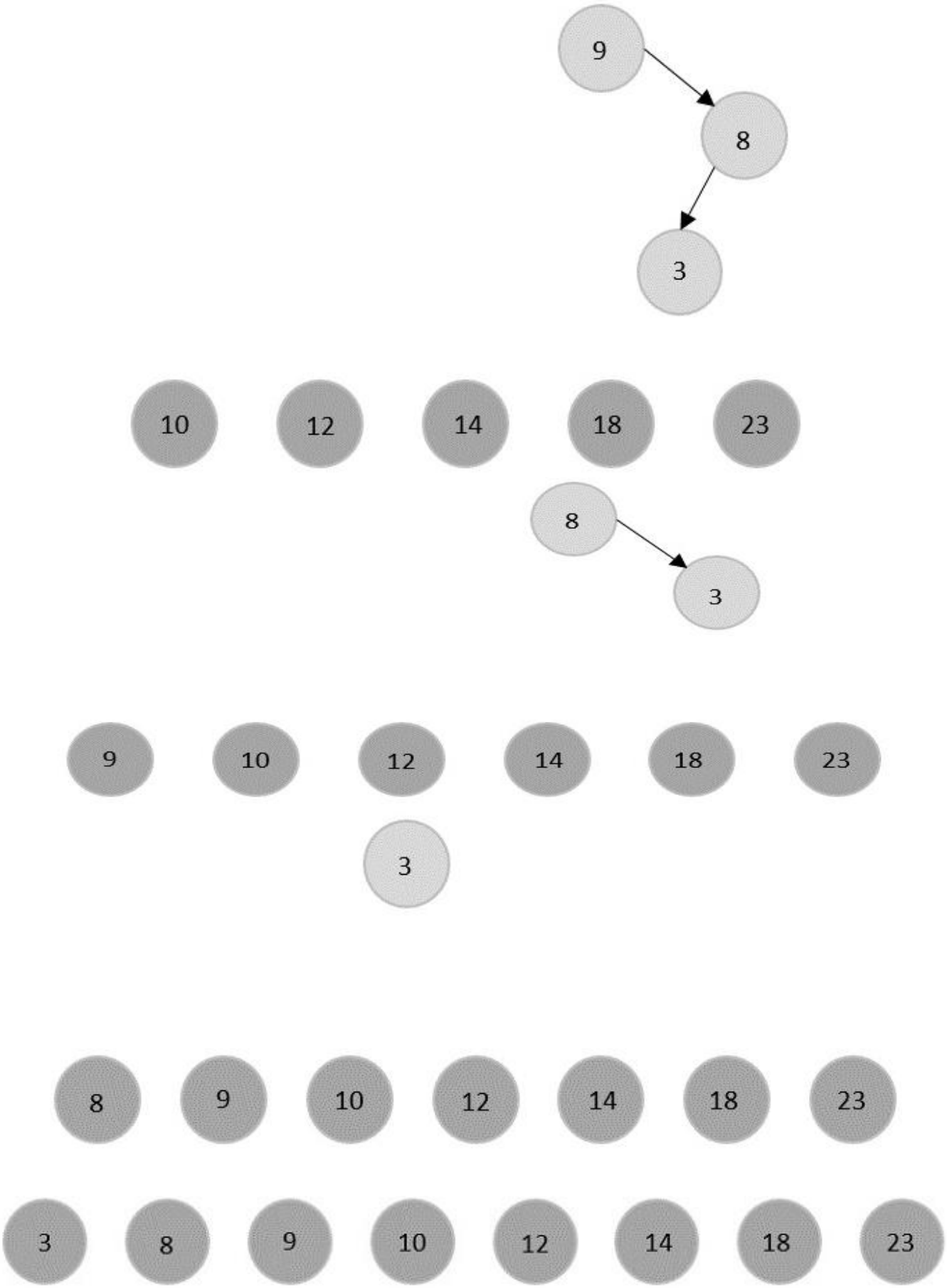
The current root 14 is popped from the heap and is replaced by 12.



12 is popped and replaced with 10.



Similarly all the other elements are popped using the same process.



Every time an element is popped, it is added at the beginning of the output array since the heap data structure formed is a max-heap. But if the heapify method converts the binary tree to the min-heap, add the popped elements are on the end of the output array.

The final sorted list is,

3 8 9 10 12 14 18 23

Implementation

The logic applied on the implementation of the heap sort is: firstly, the heap data structure is built based on the max-heap property where the parent nodes must have greater values than the child nodes. Then the root node is popped from the heap and the next maximum node on the heap is shifted to the root. The process is continued iteratively until the heap is empty.

The heap sort implementation in four different programming languages.

```
#include <stdio.h>
void heapify(int[], int);
void build_maxheap(int heap[], int n){
  int i, j, c, r, t;
  for (i = 1; i < n; i++) {
    c = i;
    do {
      r = (c - 1) / 2;
      if (heap[r] < heap[c]) { // to create MAX heap array
        t = heap[r];
        heap[r] = heap[c];
        heap[c] = t;
      }
      c = r;
    } while (c != 0);
  }
  printf("Heap array:\t");
  for (i = 0; i < n; i++)
    printf("%d\t ", heap[i]);
  heapify(heap, n);
}
void heapify(int heap[], int n){
  int i, j, c, root, temp;
  for (j = n - 1; j >= 0; j--) {
    temp = heap[0];
    heap[0] = heap[j]; // swap max element with rightmost leaf element
    heap[j] = temp;
    root = 0;
    do {
      c = 2 * root + 1; // left node of root element
      if ((heap[c] < heap[c + 1]) && c < j-1)
        c++;
      if (heap[root]<heap[c] && c<j) { // again rearrange to max heap array
        temp = heap[root];
        heap[root] = heap[c];
        heap[c] = temp;
      }
      root = c;
    } while (c < j);
  }
  printf("\nThe sorted array is : ");

  for (i = 0; i < n; i++)
    printf("\t %d", heap[i]);
}
void main(){
  int n, i, j, c, root, temp;
  n = 5;
  int heap[10] = {2, 3, 1, 0, 4}; // initialize the array
```

```
  build_maxheap(heap, n);
}
```

Output

Heap array:	4	3	1	0	2
The sorted array is :	0	1	2	3	4

Complexity Analysis of Heap Sort

Time Complexity: O(N log N)

Auxiliary Space: O(log n), due to the recursive call stack. However, auxiliary space can be O(1) for iterative implementation.

Important points about Heap Sort:

- Heap sort is an in-place algorithm.
- Its typical implementation is not stable but can be made stable
- Typically 2-3 times slower than well-implemented QuickSort. The reason for slowness is a lack of locality of reference.

Advantages of Heap Sort:

- **Efficient Time Complexity:** Heap Sort has a time complexity of O(n log n) in all cases. This makes it efficient for sorting large datasets. The **log n** factor comes from the height of the binary heap, and it ensures that the algorithm maintains good performance even with a large number of elements.
- **Memory Usage –** Memory usage can be minimal because apart from what is necessary to hold the initial list of items to be sorted, it needs no additional memory space to work
- **Simplicity –** It is simpler to understand than other equally efficient sorting algorithms because it does not use advanced computer science concepts such as recursion.

Disadvantages of Heap Sort:

- **Costly**: Heap sort is costly.
- **Unstable**: Heap sort is unstable. It might rearrange the relative order.
- **Efficient:** Heap Sort is not very efficient when working with highly complex data.